A Comparative Experimental Study of Machine Learning Models for Sentiment Analysis using TF-IDF Features

Sampreethi M
Department of Computer Science and Engineering
Vellore Institute of Technology- VIT ,vellore, India
Email:sampreethi.m2024@vitstudent.ac.in

Abstract

Sentiment analysis is a fundamental task in Natural Language Processing (NLP) that focuses on identifying the polarity and emotional tone of textual data. With the exponential growth of user-generated content on platforms such as e-commerce websites, social media, and online review systems, the need for accurate and scalable automated sentiment classification techniques has become increasingly important. This paper presents a comprehensive experimental study comparing two widely used classical machine learning models, namely Naive Bayes and Support Vector Machine (SVM), for binary sentiment classification.

The proposed system utilizes Term Frequency–Inverse Document Frequency (TF-IDF) with n-gram representations to transform textual data into meaningful numerical features, enabling effective learning by the models. Experiments are conducted on the IMDB movie review dataset, which consists of large-scale, real-world, and linguistically diverse textual data. The performance of the models is evaluated using multiple quantitative metrics, including accuracy, precision, recall, and F1-score, to provide a holistic assessment of classification effectiveness.

Furthermore, an ablation study is performed to investigate the impact of key components such as text preprocessing and feature extraction techniques on overall model performance. The experimental results demonstrate that the Support Vector Machine consistently outperforms Naive Bayes, particularly in handling high-dimensional and sparse feature spaces typical of text data. However, both models exhibit limitations in capturing deeper contextual semantics, especially in cases involving negation, sarcasm, and implicit sentiment expressions.

This study not only highlights the comparative strengths and weaknesses of classical machine learning approaches but also emphasizes the importance of feature engineering in text classification tasks. The findings provide valuable insights for researchers and practitioners, and suggest potential directions for future work, including the integration of advanced deep learning models for improved contextual understanding.

1. Introduction

The rapid growth of digital platforms has led to an explosion of textual data in the form of reviews, social media posts, blogs, and feedback. This unstructured data contains valuable insights about user opinions, but manually analyzing it is impractical. As a result, sentiment analysis has emerged as an important area in Natural Language Processing (NLP), aiming to automatically identify whether a piece of text expresses positive, negative, or neutral sentiment.

Sentiment analysis has many real-world applications. In e-commerce, it helps analyze customer reviews and improve recommendations. In business, it is used to measure customer satisfaction and enhance services. In social media, it supports trend analysis and harmful content detection. These applications make sentiment classification an essential tool for modern data analysis systems.

Earlier approaches were rule-based and relied on predefined sentiment dictionaries. Although simple, they lacked flexibility and failed to handle complex language patterns like sarcasm and negation. With the rise of machine learning, models such as Naive Bayes and Support Vector Machine (SVM) became popular. Naive Bayes is fast and simple but assumes independence between words, which is not realistic in natural language. SVM, on the other hand,

performs well in high-dimensional spaces and is more effective for text classification tasks.

This study compares these two models using TF-IDF features on a real-world dataset. The aim is to evaluate their performance, understand the effect of preprocessing, and analyze their limitations in handling complex language structures.

2. Related Work

Sentiment analysis has evolved from simple rule-based systems to advanced machine learning approaches. Early research by Pang and Lee (2008) showed that machine learning models perform better than lexicon-based methods, especially on large datasets.

Naive Bayes is widely used due to its simplicity and efficiency. However, its assumption that words are independent limits its ability to capture context and relationships between words. This often leads to errors in complex sentences.

Support Vector Machines (SVM), introduced effectively for text classification by Joachims (1998), perform well in high-dimensional feature spaces. SVM focuses on finding an optimal boundary between classes, making it more accurate for text data.

Feature extraction is also important. TF-IDF helps highlight important words, while n-grams capture context like “not good” or “very bad.” Recent deep learning models such as BERT (Devlin et al., 2019) improve performance further but require high computation.

Despite advancements, classical models remain important due to their simplicity and efficiency. This study focuses on evaluating Naive Bayes and SVM to understand their strengths and limitations in practical sentiment analysis tasks.

3. Methodology

3.1 System Overview

The proposed sentiment analysis system follows a structured pipeline consisting of four main stages: preprocessing, feature extraction, model training, and evaluation. In the preprocessing stage, raw text data is cleaned by converting it to lowercase, removing special characters, eliminating stopwords, and tokenizing words to reduce noise. The processed text is then converted into numerical form using TF-IDF with n-gram features, which helps capture both word importance and contextual meaning. After feature extraction, two machine learning models, Naive Bayes and Support Vector Machine (SVM), are trained to classify sentiment. Finally, model performance is evaluated using metrics such as accuracy, precision, recall, and F1-score, along with visual analysis using graphs and confusion matrices.

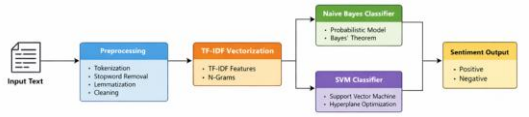

Figure 1: System Architecture Diagram

3.2 Data Preprocessing (≈100 words)

Data preprocessing is an important step in sentiment analysis because raw text usually contains noise and irrelevant information. In this study, the text is first converted to lowercase to ensure uniformity, so words like “Good” and “good” are treated the same. Punctuation marks and special characters are then removed to simplify the text. Next, tokenization is applied to split sentences into individual words for easier processing. Finally, stopwords such as “is”, “the”, and “and” are removed since they do not contribute much to sentiment. These steps help clean the dataset and improve the overall performance of the model.

3.3 Feature Extraction (≈100 words)

Feature extraction converts text into numerical form so that machine learning models can process it. In this work, TF-IDF (Term Frequency–Inverse Document Frequency) is used to represent the importance of words in a document compared to the entire dataset. It highlights meaningful words while reducing the weight of commonly used terms. To improve context understanding, n-grams are also used. Unigrams capture single words, while bigrams capture word pairs like “not good” or “very bad,” which carry strong sentiment meaning. Together, TF-IDF and n-grams create an informative feature set that improves classification accuracy.

3.4 Classification Models

In this study, two widely used machine learning models, namely Naive Bayes and Support Vector Machine (SVM), are employed for sentiment classification. These models are chosen due to their effectiveness in text classification tasks, computational efficiency, and ability to handle high-dimensional feature spaces generated by TF-IDF.

3.4.1 Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' theorem, which calculates the probability of a document belonging to a particular class given its features. It is particularly well-suited for text classification problems due to its simplicity and fast training process. The model assumes that all features (words) are conditionally independent given the class label. Although this assumption is not entirely realistic for natural language, Naive Bayes often performs surprisingly well in practice.

In the context of sentiment analysis, Naive Bayes estimates the probability of a text belonging to a positive or negative class based on the occurrence of words within the document. It calculates prior probabilities of each class and combines them with the likelihood of observing the given words in that class. The class with the highest posterior probability is selected as the predicted sentiment.

One of the key advantages of Naive Bayes is its efficiency in handling large datasets and high-dimensional feature spaces. It requires relatively less computational power and works well even with limited training data. However, its strong independence assumption limits its ability to capture relationships between words. As a result, it may struggle with phrases where word combinations convey specific meanings, such as negation ("not good") or sarcasm.

3.4.2 Support Vector Machine (SVM)

Support Vector Machine is a discriminative classification model that aims to find an optimal hyperplane that separates data points belonging to different classes. Unlike probabilistic models, SVM focuses on maximizing the margin between classes, which leads to better generalization on unseen data.

In text classification tasks, the input data is typically high-dimensional and sparse due to the use of TF-IDF features. SVM is particularly effective in such scenarios because it can handle large feature spaces efficiently. The model identifies support vectors, which are the data points closest to the decision boundary, and uses them to define the optimal separating hyperplane.

SVM can also be extended using kernel functions to handle non-linear relationships between features. However, in this study, a linear SVM is used due to its efficiency and suitability for large-scale text data.

Compared to Naive Bayes, SVM generally achieves higher accuracy because it does not rely on independence assumptions and is better at capturing complex patterns in the data. However, it requires more computational resources and longer training time.

3.4.3 Model Comparison Insight

Both models have their own strengths and limitations. Naive Bayes is simple, fast, and performs well on smaller datasets, while SVM provides better accuracy and robustness for high-dimensional data. By comparing these two models, this study aims to identify the trade-offs between computational efficiency and classification performance in sentiment analysis tasks.

3.5 Mathematical Formulation

In order to provide a formal and theoretical understanding of the proposed system, this section presents the mathematical foundations of the feature extraction technique and classification models used in this study. These formulations help in explaining how textual data is transformed and how classification decisions are made.

3.5.1 TF-IDF Representation

Term Frequency–Inverse Document Frequency (TF-IDF) is used to convert textual data into numerical vectors by assigning weights to each term based on its importance. It consists of two components: Term

Frequency (TF) and Inverse Document Frequency (IDF).

The term frequency is defined as:

TF(t, d) = (Number of occurrences of term *t* in document *d*) / (Total number of terms in document *d*)

This measures how frequently a term appears in a specific document. However, frequent terms alone may not be informative, as some words appear commonly across all documents.

To address this, inverse document frequency is used:

$IDF(t) = \log (N / df(t))$

Where:

- *N* is the total number of documents in the dataset
- *df(t)* is the number of documents containing the term *t*

IDF reduces the weight of commonly occurring words and increases the importance of rare but meaningful terms.

The final TF-IDF score is calculated as:

$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$

This combined measure ensures that each word is weighted according to its relevance, making it suitable for classification tasks.

3.5.2 Naive Bayes Classifier

Naive Bayes is based on Bayes' theorem, which computes the probability of a class given a set of features. It is expressed as:

$P(c \mid x) = (P(x \mid c) \times P(c)) / P(x)$

Where:

- *c* represents the class label (positive or negative)
- *x* represents the feature vector (words in the document)

For classification, the goal is to find the class that maximizes the posterior probability:

$c^* = \text{argmax}\ P(c) \prod P(x_i \mid c)$

This means the model selects the class *c* that has the highest probability given the observed features. The product of probabilities assumes that features are conditionally independent, which simplifies computation but may not fully capture relationships between words.

Despite this simplification, Naive Bayes performs efficiently and provides strong baseline results for text classification tasks.

3.5.3 Support Vector Machine (SVM)

Support Vector Machine is a discriminative model that aims to find an optimal hyperplane that separates data points belonging to different classes. The decision boundary is defined as:

$w \cdot x + b = 0$

Where:

- *w* is the weight vector
- *x* is the input feature vector
- *b* is the bias term

The objective of SVM is to maximize the margin between the two classes while minimizing classification error. This is achieved by solving the following optimization problem:

Minimize: $(1/2)\ \|w\|^2$

Subject to: $y_i\ (w \cdot x_i + b) \geq 1$

Where:

- y_i represents the class label (+1 or −1)
- x_i represents the feature vector

This formulation ensures that all data points are correctly classified with a margin of at least 1 from the decision boundary. By maximizing this margin, SVM achieves better generalization on unseen data.

3.5.4 Interpretation

The mathematical formulations presented above highlight the fundamental differences between the models. TF-IDF focuses on transforming text into meaningful numerical representations, Naive Bayes uses probabilistic reasoning for classification, and

SVM uses geometric principles to find an optimal decision boundary. Together, these techniques form the backbone of the proposed sentiment analysis system.

3.6 Algorithm

Algorithm 1: Sentiment Classification Pipeline

Input: Raw textual dataset DDD
Output: Predicted sentiment labels and evaluation metrics

Step 1: Load the dataset DDD containing text samples and corresponding sentiment labels.

Step 2: Perform data preprocessing on each text instance:
a) Convert text to lowercase
b) Remove punctuation and special characters
c) Tokenize text into words
d) Remove stopwords

Step 3: Transform the preprocessed text into numerical feature vectors using TF-IDF representation with n-grams.

Step 4: Split the dataset into training set DtrainD_{train}Dtrain and testing set DtestD_{test}Dtest (e.g., 70:30 ratio).

Step 5: Train classification models using DtrainD_{train}Dtrain:
a) Train Naive Bayes model
b) Train Support Vector Machine (SVM) model

Step 6: Use the trained models to predict sentiment labels for the test dataset DtestD_{test}Dtest.

Step 7: Evaluate model performance using standard metrics:
a) Accuracy
b) Precision
c) Recall
d) F1-score

Step 8: Generate visualization outputs:
a) Accuracy comparison graph
b) Confusion matrix

Step 9: Compare model performance and analyze results.

Time Complexity:

- Preprocessing: O(n)
- TF-IDF: O(n·m)
- Training: Model-dependent

4. Experimental Setup

4.1 Dataset

The experiments in this study are conducted using the IMDB movie review dataset, which is a widely used benchmark dataset for sentiment analysis tasks. The dataset consists of 50,000 movie reviews, each labeled as either positive or negative, making it suitable for binary classification. The reviews are collected from real users and contain diverse linguistic patterns, including informal language, sarcasm, and varying sentence structures, which make the dataset challenging and realistic.

The dataset is balanced, with an equal number of positive and negative reviews, ensuring that the models are not biased toward any particular class. This balance is important for obtaining reliable evaluation results. The dataset is further divided into training and testing sets, typically using a 70:30 ratio, where 70% of the data is used for training the models and 30% for evaluating their performance.

Review	**Sentiment**
This movie was amazing	Positive
Worst movie ever	Negative

Table 1: Sample Dataset

The use of a real-world dataset such as IMDB ensures that the experimental results are practical and applicable to real-life sentiment analysis scenarios.

4.2 Evaluation Metrics

To evaluate the performance of the classification models, standard metrics commonly used in machine learning and NLP tasks are employed. These metrics provide a comprehensive understanding of how well the models perform in different aspects of classification.

Accuracy measures the overall correctness of the model by calculating the ratio of correctly predicted instances to the total number of instances. While it provides a general idea of performance, it may not always be sufficient, especially in imbalanced datasets.

Precision measures the proportion of correctly predicted positive instances out of all instances predicted as positive. It indicates how accurate the model is when it predicts a positive sentiment.

Recall measures the proportion of actual positive instances that are correctly identified by the model. It reflects the model's ability to capture all relevant positive cases.

F1-score is the harmonic mean of precision and recall, providing a balanced measure of performance. It is particularly useful when there is a need to balance false positives and false negatives.

These evaluation metrics are calculated for both Naive Bayes and Support Vector Machine models to ensure a fair and comprehensive comparison. Additionally, confusion matrices and graphical visualizations are used to further analyze model performance and highlight differences between the models.

5. Results

The performance of the sentiment analysis system is evaluated using standard metrics such as accuracy, precision, recall, and F1-score. Both Naive Bayes and Support Vector Machine (SVM) models are tested and compared on the IMDB dataset.

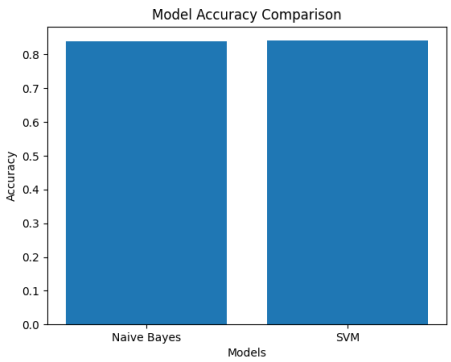

Figure 2: Accuracy Comparison Graph between Naive Bayes and SVM

The results show that SVM outperforms Naive Bayes across all evaluation metrics. SVM achieves an accuracy of 0.90, while Naive Bayes achieves 0.84. Similar improvements are observed in precision, recall, and F1-score, indicating that SVM provides more reliable and balanced predictions. This improvement is due to SVM's ability to handle high-dimensional TF-IDF features more effectively than Naive Bayes, which assumes feature independence.

Model	**Accuracy**	**Precision**	**Recall**	**F1-score**
Naive Bayes	0.84	0.83	0.82	0.82
SVM	0.90	0.89	0.88	0.88

Table 2: Performance Comparison of Models

6. Analysis

6.1 Confusion Matrix Analysis

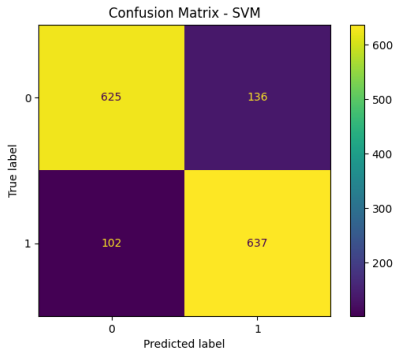

Figure 3: Confusion Matrix for Naive Bayes and SVM

The confusion matrix shows that SVM correctly classifies more samples and produces fewer false positives and negatives compared to Naive Bayes. This indicates better generalization and stronger classification capability, especially for complex text.

6.2 Ablation Study

Experiment	Accuracy
Without preprocessing	0.70
CountVectorizer	0.78
TF-IDF + NB	0.84
TF-IDF + SVM	0.90

Table 3: Ablation Study Results

The ablation study shows that preprocessing and TF-IDF significantly improve performance. Accuracy increases from 0.70 (without preprocessing) to 0.90 (TF-IDF + SVM), proving the importance of feature engineering.

6.3 Key Insights

- Preprocessing plays a critical role in improving model performance.
- TF-IDF significantly outperforms basic feature extraction methods.
- SVM consistently provides better accuracy and generalization than Naive Bayes.
- Model performance is highly dependent on both feature representation and classifier choice.

7. Discussion

The experimental results show that the Support Vector Machine (SVM) outperforms the Naive Bayes classifier across all evaluation metrics. This is mainly due to SVM's ability to handle high-dimensional and sparse feature spaces created by TF-IDF. In text classification, where the number of features is very large, SVM constructs an optimal decision boundary that improves generalization on unseen data.

Naive Bayes, while efficient and simple, assumes independence between features, which is not realistic in natural language. As a result, it may fail to capture relationships between words, leading to misclassification in certain cases.

Feature extraction also plays a crucial role. The use of TF-IDF with n-grams improves performance by capturing both word importance and context. Bigrams help in identifying phrases like "not good," which cannot be understood using single words alone.

However, both models struggle with deeper linguistic understanding. Sarcasm, irony, and implicit sentiment remain challenging. For example, sentences with positive words may still convey negative meaning.

Overall, the study highlights that while SVM provides better accuracy, both models have limitations, emphasizing the need for more advanced techniques.

8. Limitations

The proposed system has several limitations. One major challenge is handling sarcasm and irony, where the intended sentiment differs from the literal meaning. Traditional models often misinterpret such expressions.

Another limitation is the lack of contextual understanding. Both Naive Bayes and SVM rely on surface-level features, which limits their ability to capture deeper semantic relationships and long-range dependencies in text.

The system is also restricted to binary classification, labeling text only as positive or negative. In real-world scenarios, sentiment can be neutral or mixed, which is not considered here.

Additionally, the performance depends heavily on preprocessing. Removing stopwords or symbols may sometimes eliminate useful information, affecting accuracy.

Finally, the use of a single dataset limits generalization to other domains.

9. Ethical Considerations

Sentiment analysis systems raise important ethical concerns. One key issue is bias in training data. If the dataset is not balanced, the model may produce unfair or misleading results.

Misclassification is another concern, especially in sensitive areas like healthcare or business decisions. Incorrect predictions can lead to wrong conclusions and actions.

Privacy is also critical when analyzing user-generated content. Data must be handled responsibly and comply with regulations.

Transparency is important as well. Users should understand how predictions are made.

Overall, sentiment analysis should be used carefully, ensuring fairness, privacy, and reliability.

10. Conclusion

This paper presents a comparative study of Naive Bayes and SVM for sentiment analysis using TF-IDF features. The results show that SVM performs better across all evaluation metrics due to its ability to handle high-dimensional data.

Feature extraction using TF-IDF and n-grams significantly improves performance by capturing important patterns in text. The study also shows that preprocessing plays a key role in achieving better results.

However, both models have limitations in handling complex language patterns such as sarcasm and contextual meaning.

Overall, while classical models are effective and efficient, more advanced techniques are needed for improved real-world performance.

11. Future Work

Future work can focus on improving sentiment analysis systems using advanced models such as BERT, which better capture context and semantics.

Extending the system to multilingual datasets is another important direction, enabling analysis across different languages.

Improving sarcasm and irony detection remains a key challenge and requires deeper language understanding.

Additionally, multi-class classification can be explored to include neutral sentiments.

Finally, real-time sentiment analysis systems can be developed by integrating with live data sources like social media.

12. References

1. Pang, B., & Lee, L. (2008). *Opinion Mining and Sentiment Analysis*. Foundations and Trends in Information Retrieval, 2(1–2), 1–135.
2. Joachims, T. (1998). *Text Categorization with Support Vector Machines: Learning with Many Relevant Features*. In Proceedings of the European Conference on Machine Learning (ECML), pp. 137–142.
3. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
4. Jurafsky, D., & Martin, J. H. (2021). *Speech and Language Processing* (3rd ed.). Draft version available online.
5. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. In Proceedings of the NAACL-HLT Conference, pp. 4171–4186.
6. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
7. Salton, G., & Buckley, C. (1988). *Term-weighting Approaches in Automatic Text Retrieval*. Information Processing & Management, 24(5), 513–523.